Name: Nabira Khan
Roll Number: 23K-0914

DB ASSIGNMENT#2

QUESTION#01

PART A

Film	Actor	ProductionCompany
• **FilmID** *(PK)* • Title • Budget • ReleaseDate • DirectorName	• **ActorID** (PK) • Name • PhoneNumber	• **CompanyID** (PK) • Name

PART B

Main Relationships and Cardinality

1. **Film – Actor (Casting)**
    - Relationship: A Film casts many Actors; an Actor can act in many Films.
    - **Cardinality:** M:N
2. **Film – ProductionCompany (Funding)**
    - Relationship: A Film is funded by one or more Production Companies, a Production Company can fund many Films.
    - **Cardinality:** M:N

Note: Film and Director will not have any relationship defined as Director is not an entity in this scenario, it is an attribute. But for clarity: Each film has exactly one director. Where the director is not an entity but an attribute. Cardinality: 1:1

PART C

- **Associative Entity Name:** Casting
- **Attributes:**
    1. **FilmID** *(FK)* – References the Film being cast
    2. **ActorID** *(FK)* – References the Actor being cast
    3. CharacterName – The role the Actor plays in the Film

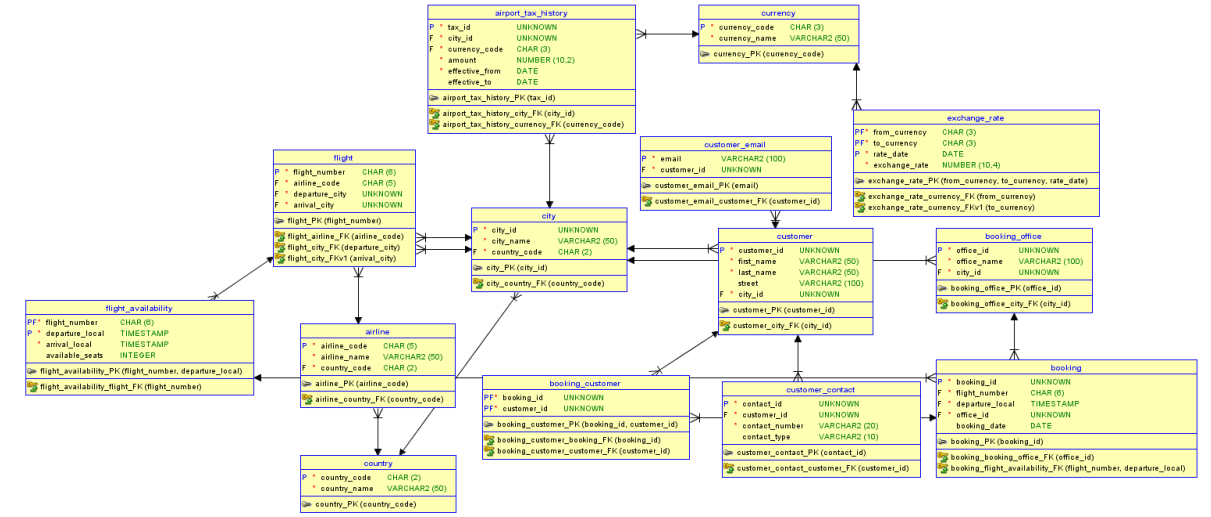
Note: (**FilmID, ActorID)** is the **primary key**, while **FilmID** and **ActorID** are foreign keys. This structure ensures each casting record is unique and allows easy extension.

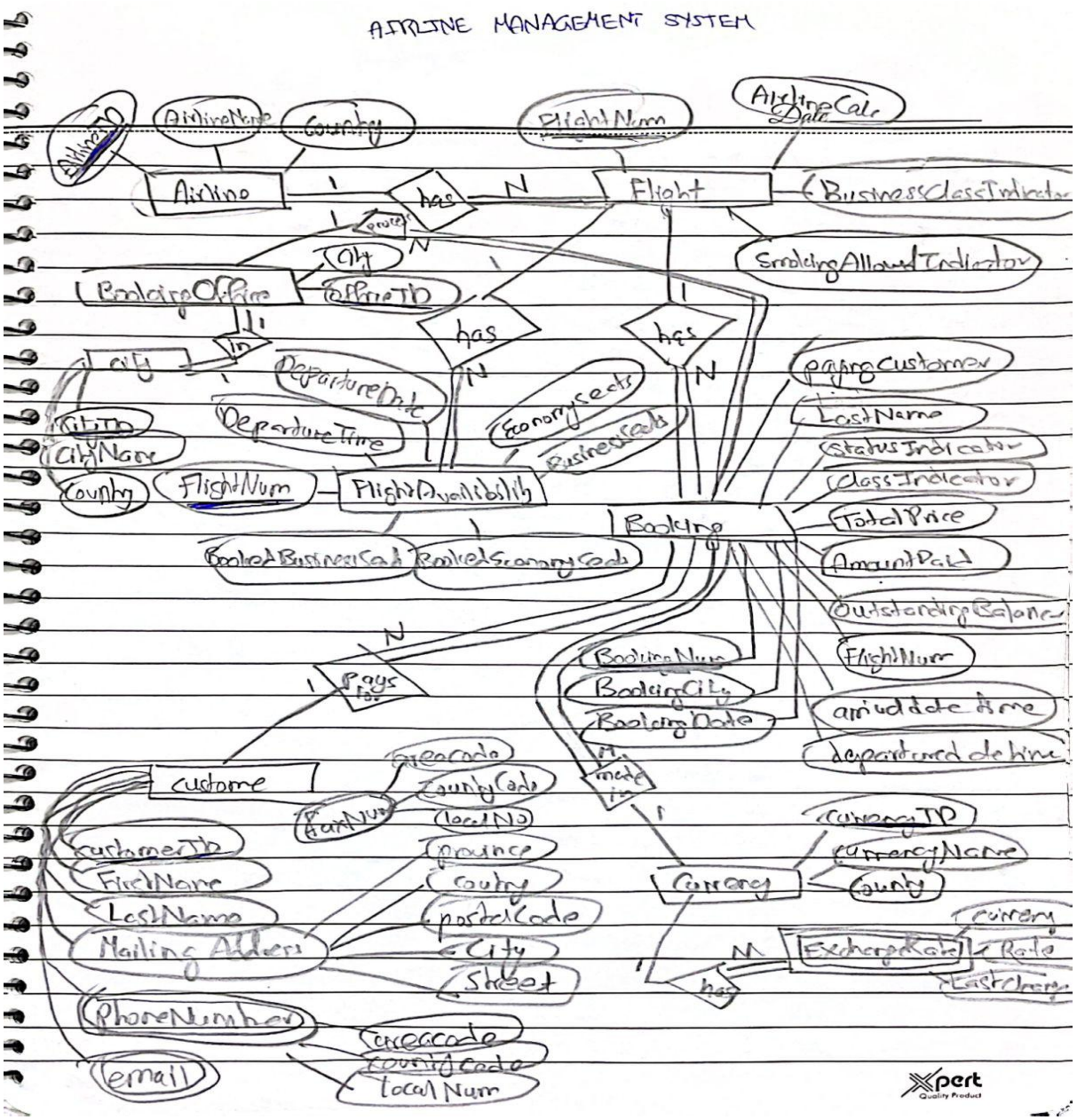
PART D

In this scenario, the **director is only tracked as a name in the Film** and no additional details about directors are required, hence it is not required as a separate entity. Creating a separate Director entity would add unnecessary complexity (extra table, foreign key, joins) without providing any additional benefit, since there's no further information to store or manage about directors.

Name: Nabira Khan
Roll Number: 23K-0914

QUESTION#02 (CAN BE DONE VIA MODELLER OR ON PAPER, I DID BOTH)





Name: Nabira Khan
Roll Number: 23K-0914

QUESTION#03

PART A

1. Insertion Anomaly

- **Reasoning:** School information (schoolID, schoolName, schoolCity) is stored together with teacher and hours info. You cannot add a new school unless at least one teacher works there.
- **Example:** To add a new school S200 Blue School London, a teacher must already be assigned; otherwise, school info cannot be stored in the table.

2. Deletion Anomaly

- **Reasoning:** School information is repeated in every row where a teacher works. Deleting a teacher record may remove the only row containing the school's information.
- **Example:** If we delete Lewis H's record for S114 Green School, the school information for Green School in Birmingham would be lost.

3. Update Anomaly

- **Reasoning:** School info is duplicated across multiple rows. Changing the school's city requires updating **all rows**; missing any row leads to inconsistent data.
- **Example:** If Park School moves from Manchester to Liverpool, all rows with S301 must be updated. If one row is missed, the table shows conflicting cities for the same school.

PART B

Functional Dependencies (FDs)

- NIN → tName (NIN uniquely identifies the teacher's name)
- schoolID → {schoolName, schoolCity} (schoolID uniquely identifies school info)
- {NIN, contractNo, schoolID} → hours (a teacher works certain hours per contract at a school)

Decompose into 1NF

- Table is already in **1NF** (all attributes atomic).

Decompose into 2NF

Partial dependencies:

- NIN → tName
- schoolID → schoolName, schoolCity

Decomposing into separate tables:

Teacher Table:

NIN	tName
2103	Brown A
2198	Khan R
2205	Lewis H

School Table

schoolID	schoolName	schoolCity

Name: Nabira Khan
Roll Number: 23K-0914

S301	Park School	Manchester
S114	Green School	Birmingham

Work Table:

NIN	contractNo	schoolID	hours
2103	T9001	S301	10
2198	T9001	S301	18
2205	T9001	S114	20
2103	T9001	S114	12

Decompose into 3NF

- All tables are now in **3NF** because:
  - No non-prime attribute depends on another non-prime attribute.
  - Each non-key attribute is fully functionally dependent on the primary key.
  - No transitive functional dependency

Final 3NF Tables:

1. Teacher (NIN → tName)
2. School (schoolID → schoolName, schoolCity)
3. Work (NIN + contractNo + schoolID → hours)

QUESTION#04

PART A

PK: (FilmID, ActorID)

PART B

Table is in **1NF** (all attributes atomic).

For 2NF:

- 2NF requires **no partial dependency** on part of a composite PK.
- **Partial dependencies exist:**
  - FilmID→ FilmTitle, DirectorName (depends only on part of PK)
  - ActorID→ ActorName, AgentID (depends only on part of PK)
  - **Violation: 2NF**, because non-key attributes depend on **part of the composite key**, not the whole key.

Example: FilmTitle depends only on FilmID, not on ActorID.

PART C

Removing partial dependencies on the composite PK.

1. **Film Table**
   - **Attributes:** FilmID *(PK)*, FilmTitle, DirectorName
   - **FD Removed:** FilmID → FilmTitle, DirectorName
2. **Actor Table**
   - **Attributes:** ActorID *(PK)*, ActorName, AgentID
   - **FD Removed:** ActorID → ActorName, AgentID
3. **Film_Casting Table (2NF)**

- **Attributes:** FilmID *(FK)*, ActorID *(FK)*
- **FD Remaining:** FilmID + ActorID → (all other attributes removed to separate tables)

PART D

Removing transitive dependencies (non-key attributes depending on other non-key attributes).

1. **Film Table** (already in 3NF)
   - **Attributes:** FilmID *(PK)*, FilmTitle, DirectorName
2. **Actor Table**
   - **Attributes:** ActorID *(PK)*, ActorName, AgentID *(FK)*
3. **Agent Table**
   - **Attributes:** AgentID *(PK)*, AgentPhone
4. **Casting Table**
   - **Attributes:** FilmID *(FK)*, ActorID *(FK)*

All transitive dependencies are removed: AgentPhone depends only on AgentID, not on ActorID or FilmID.

PART E

Data Redundancy Anomaly

- **Type of anomaly:** Update Anomaly

Example:

- ActorID = 101 has AgentID = A10 with phone 555-1234.
- If Actor 101 acts in 3 films, AgentPhone appears 3 times. Changing the phone requires updating **all 3 rows**; missing one causes inconsistent data.

QUESTION#05

PART A

Attributes from the form:

- StudentID, LastName, FirstName, Address
- CourseID, CourseName, Credits, InstructorName
- EnrollmentDate

Assumptions:

1. Each **StudentID uniquely identifies a student**.
2. Each **CourseID uniquely identifies a course**.
3. Students can enroll in multiple courses; courses can have multiple students.
4. InstructorName depends on CourseID (each course has one instructor).
5. EnrollmentDate depends on StudentID + CourseID (date student enrolled in that course).

Functional Dependencies:

1. StudentID → LastName, FirstName, Address
2. CourseID → CourseName, Credits, InstructorName
3. (StudentID, CourseID) → EnrollmentDate

Name: Nabira Khan
Roll Number: 23K-0914

PART B

1NF:

- Table is already in 1NF (atomic values, no repeating groups).

2NF:

- **Composite key:** (StudentID, CourseID) in the enrollment table
- **Partial dependencies exist:**
  - StudentID → LastName, FirstName, Address
  - CourseID → CourseName, Credits, InstructorName
- **Removing partial dependencies** by creating separate tables:

Students Table:

StudentID *(PK)*	LastName	FirstName	Address
S1001	Smith	James	123 Oak Street

Course Table:

CourseID *(PK)*	CourseName	Credits	InstructorName
CS101	Introduction to Computer Science	3	Dr. Baker
MA202	Calculus II	4	Dr. Johnson
PH301	Physics	3	Dr. Lee

Enrolment Table:

StudentID *(FK)*	CourseID *(FK)*	EnrollmentDate
S1001	CS101	15/01/2024
S1001	MA202	15/01/2024
S1001	PH301	15/01/2024

3NF:

All tables are already in **3NF** because:

- Non-key attributes are fully functionally dependent on the PK of their table.
- No transitive dependencies exist.

PART C

Table	Primary Key (PK)	Alternate Key(s)	Foreign Key(s)
Student	StudentID	None	None
Course	CourseID	CourseName (assumed unique)	None
Enrollment	(StudentID, CourseID)	None	StudentID → Student, CourseID → Course